↖ Topic Index

File Management

Operation	Keystroke	Function	Note

File Handling
See also: Σ **Dired**
• Σ **Customize**
• Σ **Key-Chords**
• Σ **Tramp** (edit remote)
Features:
• open file
○ open file at point
○ ffap commands
• Recently opened
• open modes (read-only, root, binary)
○ Fuzzy file finder
• Dired (directory editor)
• Activate URLs in buffer
• Insert text from file
○ Write text to file
• Reverting file
• Save to file, Rename
• Inserting copyright
○ Automatic time stamp
○ RFC mode
• Directory tree browsers
• Treemacs/ZTree
○ Search/Find Files
 	
Last updated on:
			Emacs provides a large set of commands to open files (Emacs documentation uses the term "*finding*" files for that), saving files searching for files or file content, displaying directory content, etc… These are listed in this table. The directory editing (dired) commands are mainly listed in the Σ **Dired** table.
			• There are also several Emacs internal and external packages that provide useful commands. PEL supports several of them, listed below.
			  • Use Emacs customize system to modify their values to activate, deactivate and modify the behaviour of these packages.
			  • PEL **<f11> f** key prefix followed by either **<f2>** to access PEL activation group and **<f3>** to access the external package customization groups.
			  • Once you have modified the relevant user-option values, apply or save them and then either execute **`M-x pel-init`** or restart Emacs.
			PEL provides integration with the following Emacs built-in libraries or functionalities:
			• 📦 **archive-rpm** activated by **pel-use-archive-rpm**, provides ability to open **RPM** and **CPIO** archive files as you can do with **tarball** and **zip** files.
			• Library **ffap** activated by **pel-use-ffap** to provide several commands to open file at point.
			• Library **recentf** activated by **pel-use-recentf** to list files recently opened.
			• 📦 The **fzf.el** external package activated by **pel-use-fzf** to provide fast fuzzy finder using fzf. See **fzf manual**, **fzf search syntax**.
			• **Automatic file time stamp** update on file save activated by **pel-update-time-stamp**.
			• **Automatic update of copyright notice** year on file save activated by **pel-update-copyright**.
			• It also provides integration with the following external packages when the corresponding PEL user-option is activated:
			  📦 **`key-chord`** activated by **pel-use-key-chord**, provides convenient key-chords for some commands.
			  📦 **`rfc-mode`** activated by **pel-use-rfc**, provides ability to download and browse **IETF** **RFC** documents easily (see **RFC editor**).
			  📦 **`ivy/counsel`** activated by **pel-use-counsel** provides completion for some file commands. PEL supports more. See Σ **Completion/Input**.
			  📦 **`NeoTree`** activated by **pel-use-neotree** provides an alternative to Σ **Dired** to navigate a file directory.
			  📦 **`treemacs`** activated by **pel-use-treemacs** provides project-oriented file directory navigation. See Σ𝑥 **Treemacs**
			  📦 **`ztree`** activated by **pel-use-ztree an other** alternative to Σ **Dired** to navigate a file directory.
			
			2026-03-15

Open this PDF file. See also: Σ **Help/Info**	**`<f11> f <f1> 1`**	(**pel-help-pdf** &optional OPEN-WEB-PAGE)	Open the Σ **File-mngt** local PDF. If the prefix argument (like **`C-u`** or **`M--`**) is used, then open remote GitHub hosted raw PDF instead. If **pel-flip-help-pdf-arg** user-option is set it's the other way around.
Σ **Customize PEL File/ Directory Management**	**`<f11> f <f2> 1`**	(**pel-customize-pel** &optional OTHER-WINDOW)	Customize PEL support for file management. • If OTHER-WINDOW is non-nil (use **`C-u`**), display in other window.
Σ **Customize Emacs file management support**	**`<f11> f <f3>`**	(**pel-customize-library** &optional OTHER-WINDOW)	Customize Emacs support for file management. Includes the following: files, fzf, recentf, popup-switcher, x509 (see **`M X.509 Certificates`**).
Customize Emacs support for file revert	**`<f11> f r <f3>`**	(**pel-customize-library** &optional OTHER-WINDOW)	Customize Emacs support for file automatic revert management.
Customize ffap (find file at point)	**`<f11> f a <f3>`**	(**pel-customize-library** &optional OTHER-WINDOW)	Customize Emacs support for management of ffap (find file at point).
Show file mngt status	**`<f11> f ? ?`**	(**pel-show-filemng-status**)	Display status of various file management controls: encoding, resolving relative path method, etc..
Show RPM distributing current file	**`<f11> f ? r`**	(**pel-show-rpm-providing-file**)	Display the name of the RPM that distributes the file opened in the current buffer. 🐧 Available on Linux systems only. See **RPM Files**
Open File in App	The following command opens file(s) outside Emacs, using OS applications registered with the file type. See: Σ **Dired**, Σ **Web**		
Open currently file visited in current buffer with the default OS application.	**`<f11> f C-o`**	(**pel-open-buffer-file-in-os-app** &optional FNAME)	Open the current buffer file with OS-registered application. If buffer modified, prompt to save buffer first. ☝ If the current buffer holds a HTML file, that's a quick way to open the file in your browser. • In dired-mode buffers: open each marked files in its S-registered applications, you can also type **`z`** to open the current file or all selected files.
Opening file See also: • Σ **Completion/Input** • Σ **Dired** • Σ **Tramp** (edit remote computer files) • **Typing file name in minibuffer**	The following commands are available to open/visit files in Emacs buffers. **Note**: Emacs uses the word "*visiting*" instead of "*opening*" files. • For some of them the corresponding ido mode function is also shown. • The command used to 'visit' a file, find-file is Emacs default. It supports Emacs' basic tab completion. Packages that support other completion mechanisms can be installed and activated and then the command uses a different completion mechanism. • ☝PEL customization system allows you to specify whether you want to use one or several other completion mechanisms. It also has a command to change the completion mechanism dynamically. You can change it without restarting Emacs or event re-executing pel-init. • See the Σ **Completion/Input** and Σ **Customize** tables for more info. **File Lock :** Emacs protects against multiple processes modifying the same file with a lock. If you attempt to edit the buffer of a locked file, or save a buffer of a locked file , Emacs will prompt. You can then: 1) steal the lock (with 's'), 2) proceed ('p') to edit the file anyway or 3) quit ('q').		
Open file-open dialog	**`⌘-o`**	(**ns-open-file-using-panel**)	🍎 On macOS in graphics mode only: open a file, select the file name via an OS File dialog.
Open (visit) a file/ directory	• **`<f11> f f`** • **`C-x C-f`**	(**find-file** FILENAME &optional WILDCARDS)	Prompt for the file or directory name to open. Open the selected file/directory in a buffer with the appropriate mode. For directory, the buffer opens in Dired-mode. • The PEL **`<f11> f f`** is always available, regardless of what completion mechanism is in use.
		(**ido-find-file**)	Same as above with Ido completion. See Σ **Completion/Input** for available completion modes.

• To open file as sudo: **`C-x C-f /sudo::/ path/to/file`**
• To open file as su: **`C-x C-f /su::/ path/to/file`**
• Prevent Ido expansion with **`C-f`** ☛
• Open in read-only ☛
• Change input completion method ☛
	• **find-file** is the original command and uses Emacs default completion. When Ido is used, the **ido-find-file** command is used instead.
	• When **ido** mode is used, you can also:
	  • Type **`C-f`** or **`C-x f`** to change to original find-file mode and prevent Ido completion from trying to provide the name of an existing file when you want to specify the name of a file that does not exists yet.
	  • Type **C-j** to accept the file/directory name verbatim without replacement or suggestion. Also useful to open a directory in dired mode.
	  • To open a file in read-only mode you can: Use one of the commands below (**`C-x C-r`**, etc…)
	  Use **`C-x C-f`** then type **`C-x C-q`** to change the mode of the buffer to read-only mode.
	  • Control whether it opens file at point is opened by **ido-use-filename-at-point** user-option. Use **`<f11> f M-.`** to dynamically change it.
	  • Control whether it opens url at point by **ido-use-url-at-point** user-option. Use **`<f11> f M-,`** to dynamically change it.
	Use **`<f11> M-c <f4>`** to select another input completion method. See Σ **Completion/Input**.		

Open file via popup menu	**`<f11> f M-f`**	(**pel-psw-navigate-files**)	Open file from a pop-up menu listing files in current directory. Uses (**psw-navigate-files "."**). • Narrow menu list by typing part of the file name. You can also select directory names. 📦 Requires **popup-switcher** PEL activates when **pel-use-popup-switcher** is **t**.
Open another file in buffer	**`C-x C-v`**	(**find-alternate-file** FILENAME &optional WILDCARDS)	Kills buffer and open the newly specified file in a new buffer same window. • When ido-mode is used, the ido-find-alternate-file is used instead. • Useful when just selected an empty file just selected by mistake.
		(**ido-find-alternate-file**)	
Open file in other window	• **`C-x 4 f`** • **`<f11> f o`**	(**find-file-other-window** FILENAME &optional WILDCARDS)	Edit file FILENAME, in another window.
		(**ido-find-file-other-window**)	• Like **`C-x C-f`**, but creates a new window or reuses an existing one.
Open file in other frame	**`C-x 5 f`**	(**find-file-other-frame** FILENAME &optional WILDCARDS)	Edit file FILENAME, in another frame.
		(**ido-find-file-other-frame**)	• Like **`C-x C-f`**, but creates a new frame or reuses an existing one.
Open same file in other directory ☝ Use it to open same file in other repo	• **`<f11> f M-d`** • **`M-<f11> M-f M-d`**	(**pel-open-file-in-other-dir**)	Open file of same name as current one present in another directory.
		• First prompt with the name of the directory of currently visited file using the default completion mechanism ('ido' by default). • Use the prompt to select the name of the other directory (which must already exist). • Use **`C-f`** to edit the dir path without completion. Select dir name, hit **`<RET>`** to open the same file in the selected other directory.	
Open file with alternate extension	**`M-<f11> M-f M-f`** **`M-<f12> M-f`**	(**pel-open-file-alternate**)	Open a file with same name but an alternate extension (depends on the language) 🎛The pel-file-extension-alist user option defines the extension pairs. If the alternate file is not found, save the file basename in the kill ring and prompt for the file name to open. • The **`M-<f12> M-f`** is available in some modes (C, C++, Objective-C) but The global one can be used in Ada, Seed7.

Operation	Keystroke	Function	Note
Set whether ido-find-file uses filename at point See also: **Completion/ Input**	`<f11> f M-.`	(**pel-set-ido-use-fname-at-point** &optional GLOBALLY)	Enable or disable Ido ability to open URL at point with **`C-x C-f`** and other ids commands. • Control behaviour in local buffer by default. Use command prefix to control it globally. • This is not persistent. User option **ido-use-file-at-point** controls persistent setting. Set it to one of: disabled : don't use filename at point. • guess : try to identify an exiting file name from the name at point. • literal : use name at point in the Ido search for a file name.
Set whether ido-find-file uses URL at point	`<f11> f M-,`	(**pel-set-ido-use-url-at-point** &optional GLOBALLY)	Enable or disable Ido ability to open URL at point with **`C-x C-f`** and other ids commands. • Control behaviour in local buffer by default. Use command prefix to control it globally. • This is not persistent. User option **ido-use-url-at-point** controls persistent setting.
Open file at point	The following commands, open files from the file name taken at point (the cursor location). They work regardless of the current input completion method. ☝ Note that when using the Ido completion mode, it is possible to instruct Ido to use a file name at point as the basis for the file name to open. This Ido behaviour is controlled by the **ido-use-filename-at-point** user-option. With PEL you can control it globally or locally with **<f11> f M-.**		
Set base directory for pel-open-at-point relative file names ☝ Use it to set a base directory to a remote host directory and **open remote host files easily**!	`<f11> f ;`	(**pel-set-open-at-point-dir**)	Set the behaviour of '**pel-open-at-point**' in **current buffer**. Which defaults to value selected by **pel-open-file-at-point-dir** user-option. • Select method used to determine the directory from which a relative file name is built from following methods: • Use visited file parent directory (the default). • Use buffer's current working directory. • Use specified directory. Prompts for the directory name. • Supports completion and ☝ **Tramp** syntax to setup the base directory to a remote host directory: • **pel-open-at-point** will then open remote host files! 😇

Open local/remote file or web-page whose name is at point
★★
Command is generic and is also specialized for:
• **reStructuredText**
• **C** , **C++**
• **Erlang**
• **Perl**
• **UNIX Shell**
See **Tramp**
Delimiting characters ➡
File identification heuristic ➡
`<f11> f <f2>` ➡
`<f11> f ;` ➡
Select multi-file selection method ➡
Select target window ➡
N>20 : open the directory ➡
See function docstring for more info.
	• `M-*`
	• `<f11> f .`
	• `6y`
		(**pel-open-at-point** &optional N)	Open the file, library or the URL, named at point, with potential line & column #s.
			• If necessary will search source code files in current project as specified by **pel-filename-at-point-finders** user-option. Type **<f12> <f4> ?** to **show used file search method** in supporting modes.
			🎛 Supports glob characters, partial directory path. When multiple files are found it prompts using the method selected by **pel-prompt-read-method** user-option.
			📦 The **6y** key-chord is available if **pel-use-key-chord** is non-nil. See **Key-Chords**.
			☝ This command works generically in all buffers but is also specialized for some major modes, like C, C++, Erlang, Perl, reStructuredText, UNIX Shell.
			• When executed from with a buffer in sh-mode, the '=' and ':' characters are used as additional delimiters. Expands shell variables (such as $HOME).
			• The logic supports the **Tramp** remote file syntax if it's present, allowing the opening of remote files by text at point. For normal file names, the characters `:`, `@` and `#` are not allowed in file names for most modes (but ':' is allowed in Perl major modes, for example).
			In general the command extracts the file or directory name, and possibly line and column numbers, from text at point and tries to open the file or directory.
			• The generic mode extraction works by identifying the beginning & end of the file/directory/library/URL name string by delimiter characters, one of: tab, newline and:
			`"`'`|()[]{}<>` ' ' " " 「」 〔〕 〈〉 《》 【】 〖〗 «» ‹› ❮❯ ❬❭ 〘〙 ·。
			• If embedded space(s) are allowed in the name, point must be located at the first of the 2 delimiter chars. Otherwise point can be anywhere in the name.
			• The name may include glob characters (but not in C/C++ in `#include ""` or `#include <>` statements).
			The command uses a URL unchanged but uses the following heuristic to identify the exact location of the file/directory:
			• In the file/dir name is an absolute path it uses that. Otherwise
			• it builds a absolute path using the extracted relative path name inside the directory identified by the **pel-open-file-at-point-dir** user-option, which can be 1) use parent directory of currently visited file, or use current working directory, 2) use current working directory, or 3) use user-specified directory. It uses the found file/dir name if it exists. Otherwise
			• it searches for the relative file/dir name in directory tree under the root marker file identified by the **pel-project-root-identifiers** user-option which is something like .git, .hg, .project, .pel-project (the default). If it can find such a file in the above directories it searches the tree under the found root.
			• If it finds several files it prompts using the current completion mode to allow selection of the appropriate name (see below) and opens the selected one.
			• If it finds only one it opens that file. Otherwise,
			• it prompts showing the name searched and provide the following choices: 1) create the file with specified name, 2) edit the name to search again, 3) use the name found and search for an Emacs library file with that name, or 4) quit.
			The command **opens the extracted name according to this heuristic**:
			• If the string is a properly formatted **URL**, it opens it using the OS default browser (even if a optional numeric argument specified otherwise), otherwise
			• if the string is a file or directory name it opens it.
			• If the file name is followed by **line and column numbers** the point is moved to that position in the buffer.
			🎛 When finding several file names, the command lists them and prompts using the method selected by **pel-prompt-read-method** user-option.
			• The default is a very primitive function implemented by PEL. You can select a more powerful **ivy** prompting instead.
			• With **ivy** selected, PEL will automatically set **pel-use-ivy** to **t** 📦 and **Ivy mode** will be installed automatically when you restart Emacs.
			• Note that the command shows all files found by the specified search method, it does not only use the first one found.
			• ☝ Use this to detect potential duplication in header file names in large include paths.
			The command **opens the file in the window selected** by the following logic controlled by presence or absence of typed numerical prefix arguments:
			• **Select target window:**
			• Without argument:
			• If file or directory is already opened in a window, move point to that window and to the line column coordinates if specified.
			• If no window holds that file, select the target window according to the number of editable windows in frame: if 1, split that window and use the new window, if 2: use the other window, if 3 or more, use the current window.
			• With prefix numeric argument N:
			• N < 0 : create a new window and use that.
			• (abs N) > 20: then open the **directory** instead of the file. Interpret the window position from the N value adjusted: N-20 (or N+20 if N is negative)
			• N = 0: use the '*other*' (the next) window.
			• N = 1, 3, 7or above (excluding 8, 9 and 10): select the target window based on the number of editable windows in frame:
			• if 1 window: split that window and use the new window,
			• if 2 windows: use the other window,
			• if 3 or more windows: use the current window.
			• N is: 8: up, 2: down, 4:left, 5:current, 6:right.
			• *on a numerical keyboard the location of the numeric key represent direction* ➡
			`8 := up`
			`4 := left` `5 := current` `6 := right`
			`2 := down`
			• N is 9: force **opening the file in the OS associated application** (with N=29 or N=-29, open the file's directory with the OS associated application (eg. macOS Finder, Windows Explorer). If this is a URL, open it in the OS default web browser.
			• Selecting Minibuffer, inexistent or dedicated window is not allowed.	

Open filename at point in a browser See also: **Key-Chords** , **Web**	• `<f11> f /` • `6u`	(**pel-browse-filename-at-point**)	Open the file name (or URL) at point inside the system's web browser. • If point is at dir name, open the OS app. browsing dirs (eg. macOS Finder, Windows Explorer). ☝ This is the same as using pel-open-at-point with the argument N set to 9. It is easier to type and PEL assigns its own key-chord for it.
Open URL at point in a browser See also: • **Key-Chords** , • **Web**	• `<f11> f M-/` • `7u`	(**browse-url-at-point** &optional ARG)	Ask a WWW browser to load the URL at or before point. • Variable 'browse-url-browser-function' says which browser to use. • With prefix argument inverts the value of the option 'browse-url-new-window-flag'. ☝ Use `<f11> <f2> E u` to open the browse-url group that contains relevant user options.
Copy URL at point in temporary file and visit the file	`<f11> f M-u`	(**pel-open-url-at-point**)	Copy the URL at point to a local temporary file and visit that file. • ⚠️The download copy of the file does not have the same name and may not open with the proper mode because it won't have an extension. The HTML formatted files will be recognized by Emacs but most of the files won't be. • Save the file somewhere else using the **C-x C-w** key sequence and identify the proper extension to activate the required major mode.
• With **goto-address-mode**	`C-c C-f`		☝ This binding is only available when point is over the URL and the **goto-address-mode** minor mode is active. Use **<f11> f u** or **<f11> f U** to activate this mode.
Show file name extracted from text	`<f11> f ? n`	(**pel-show-filename-at-point**)	Display file name extracted at point in the mini-buffer. Testing utility.
Show file name parts extracted from text at point	`<f11> f ? N`	(**pel-show-filename-parts-at-point** &optional KEEP-FILE-URL)	Display file parts extracted from point. Testing utility.

Operation	Keystroke	Function	Note
ffap commands	Emacs provides the ffap (find file at point) command set. The ffap command is similar to pel-find-file-at-point-in-window but does not support line and numbers, does not support identifying a window with command arguments and is not designed to support multiple programming languages. It does however support other facilities and can be installed to replace the behaviour of standard file management command bindings such as **C-x C-f**. PEL activates the Emacs built-in ffap library when the **pel-use-ffap** user option is set to either **t** or to **ffap-bindings**. In both cases these activate the key bindings shown below. • When **pel-use-ffap** is set to ffap-bindings, then PEL also activates the standard ffap bindings which take over the behaviour of the main file finding and dired commands. This means that Ido, Ivy or Helm are no longer available for these commands. • If **pel-use-ffap** is only set to **t** then the standard ffap bindings is not activated.		
Find file/URL at point	**<f11> f a p**	(**ffap** &optional FILENAME)	Find FILENAME, guessing a default from text around point. • If 'ffap-url-regexp' is not nil, the FILENAME may also be an URL. Web URL opens in browser. • With a prefix, this command behaves exactly like 'ffap-file-finder'. • If 'ffap-require-prefix' is set, the prefix meaning is reversed. • See also the variables 'ffap-dired-wildcards', 'ffap-newfile-prompt', 'ffap-url-unwrap-local', 'ffap-url-unwrap-remote', and the functions ffap-file-at-point' and 'ffap-url-at-point'.
Find file/URL at point - read only	**<f11> f a P**	(**ffap-read-only**)	Like 'ffap', but mark buffer as read-only.
Find another file/URL at point in window	**<f11> f a v**	(**ffap-alternate-file**)	Like 'ffap' and 'find-alternate-file': kills current buffer and open new file in the same window.
Find file/URL in other window	**<f11> f a w**	(**ffap-other-window**)	Like 'ffap', but put buffer in another window.
Find file/URL in other frame	**<f11> f a f**	(**ffap-other-frame**)	Like 'ffap', but put buffer in another frame.
Find file/URL in other window - read only	**<f11> f a W**	(**ffap-read-only-other-window**)	Like 'ffap', but put buffer in another window and mark as read-only.
Find file/URL in other frame - read only	**<f11> f a F**	(**ffap-read-only-other-frame**)	Like 'ffap', but put buffer in another frame and mark as read-only.
Start Dired with file at point	**<f11> f a d**	(**dired-at-point** &optional FILENAME)	Start Dired, defaulting to file at point. See 'ffap'.
Start Dired with file at point in other window	**<f11> f a D**	(**ffap-dired-other-window**)	Like 'dired-at-point', but put buffer in another window.
Start Dired with file at point in other frame	**<f11> f a M-d**	(**ffap-dired-other-frame**)	Like 'dired-at-point', but put buffer in another frame.
List directory of file at point	**<f11> f a l**	(**ffap-list-directory**)	Like 'dired-at-point' and 'list-directory'.
Open a menu of all files, URL in current buffer.	**<f11> f a m**	(**ffap-menu** &optional RESCAN)	Put up a menu of files and URLs mentioned in this buffer. Set mark, jump to choice, and try to fetch it. The menu is cached in 'ffap-menu-alist', and rebuilt by 'ffap-menu-rescan'. • With prefix argument: forces a rebuild. Searches with 'ffap-menu-regexp'.
Recently opened • Σ **Completion/Input**	When the **pel-use-recentf** user option is set to **t**, PEL ensures that Emacs remembers the list of recently opened files and provides: • the **pel-initial-recent-f-function** user-option identifies which function use used to open the recently opened files: • ido-recentf-open : uses the current Ido prompt or Ido enhanced mechanism. Use **<f11> M-c ?** to list them and see which one is active. • counsel-recentf : uses a vertical list prompt. Requires **counsel** external package activated by **pel-use-counsel** • psw-switch-recentf : uses a popup menu • The menu bar includes a **File->Open Recent** menu entry. Some other functions are activated by their respective user options.		
Open recently opened files, using active method	**<f11> f M-r M-r**	(**pel-find-recent-file**)	Open the recent file prompt using the currently active function. • The function is selected by **pel-initial-recent-f-function**. Change with pel-select-recentf-function, bound to **<f11> f M-r M-R**. • When basic Ido is used, type **<tab>** to get possible expansions listed in a separate buffer. • Ido completion is selectable. Use **<f11> M-c ?** to list them and see which one is active. • When counsel-recentf is used, you can type **C-c C-o** to copy the list of files inside a special buffer.
Display the name of the function used to prompt for recently opened file	**<f11> f M-r M-?**	(**pel-show-recentf-function** &optional AFTER-SELECTION-P)	Display what function is used to visit recently opened files. • The argument is for internal use, it is not available interactively.
Select the function used to list/prompt the recently opened files.	**<f11> f M-r M-R**	(**pel-select-recentf-function** &optional RECENTF-FUNCTION SILENT)	Select the function to visit recently opened files. Modifies what is used in the current editing session, not the persistent value selected by the **pel-initial-recent-f-function** user-option. • The arguments are for internal use, they are not available interactively.
Edit list of recently opened files	**<f11> f M-r M-e**	(**recentf-edit-list**)	Show a dialog to delete selected files from the recent list. • Use this to remove some of the files from the list.
Open a recently opened file searched by fzf	**<f11> f M-r M-z**	(**fzf-recentf** &optional WITH-PREVIEW)	Open a recently opened file selected by fzf search. With **C-u** show file preview. See fzf below. Requires the **fzf.el** external package activated by **pel-use-fzf** .
Open in read-only	The following commands open files in read-only mode. While in read-only mode, use Use **C-x C-q** to permit editing.		
Open a file in read-only mode	**C-x C-r**	• (**find-file-read-only** FILENAME &optional WILDCARDS) • (**ido-find-file-read-only**)	Edit file FILENAME but don't allow changes. Like **C-x C-f**, but marks buffer as read-only. Use **C-x C-q** to permit editing.
Open file in other window in read-only mode	• **C-x 4 r** • **<f11> f O**	• (**find-file-read-only-other-window** FILENAME &optional WILDCARDS) • (**ido-find-file-read-only-other-window**)	(find-file-read-only-other-window FILENAME &optional WILDCARDS) Edit file FILENAME in another window but don't allow changes. Like **C-x C-f**, but marks buffer as read-only. Use **C-x C-q** to permit editing.
Open as root	On Unix/Linux/macOS some files are write protected and can only be opened with root privilege with su or sudo. Use the following command for those. Use Σ **Tramp** syntax to open a file as sudo with: **C-x C-f /sudo::/path/to/file**, as su with: **C-x C-f /su::/path/to/file**		
Open file with root privilege	**<f11> f R**	(**pel-edit-as-root** &optional ARG)	Open a file as root with sudo. Prompt for password if necessary. • If already visiting a file and a prefix ARG is specified then edit currently visited file as root.
Open Literally	Open a file with no encoding conversion: file is opened in the Fundamental mode: the major mode normally associated with the file type is not used. Note that when using Ido completion, it is possible to use a command during completion to force Ido to open the file literally. However, if you are using Emacs default completion, the following command is the only way to open a file literally.		
Visit a file literally: with no encoding support and conversion	**<f11> f M-l**	(**find-file-literally** FILENAME)	Visit file FILENAME with no conversion of any kind. • Format conversion and character code conversion are both disabled, and multibyte characters are disabled in the resulting buffer. • The major mode used is Fundamental mode regardless of the file name, and local variable specifications in the file are ignored. • Automatic uncompression and adding a newline at the end of the file due to 'require-final-newline' is also disabled. • If Emacs already has a buffer which is visiting the file, this command asks you whether to visit it literally instead.
Open binary	Open a file in hex binary mode. There are also commands to convert current buffer to hexadecimal editing, like nhexl (described in Σ **Buffers**).		
Open a file in hexl-mode See also: Σ **Buffers**	**<f11> f M-x**	(**hexl-find-file** FILENAME)	Edit file FILENAME as a binary file in hex dump format, using the 'hexl-mode'. • Switch to a buffer visiting file FILENAME, creating one if none exists.

Operation	Keystroke	Function	Note
Fuzzy File Finders See fzf manual, fzf search syntax.	The fzf command line utility is a very fast fuzzy file finder that can be used within Emacs via the fzf.el emacs front-end. To use it inside Emacs, you must: 1. install and configure the fzf command line utility, and use one of the following package to use the corresponding commands: 1. 📦 the fzf.el external package activated by **pel-use-fzf** . The fzf commands below are available when this is active. 2. 📦 the ivy/counsel external package activated by **pel-use-counsel** . The counsel commands below are available when this is active.		
Open file searched by fzf in current directory	• `<f11> M-z M-z` • `<f11> f z`	(**fzf** &optional WITH-PREVIEW)	Open a file selected by fzf session in the current directory. Type partial file name, use fzf filter expressions. Select one file and hit return to open it inside current window. • Process current working directory or Projectile process root directory if available.
☝ **fzf & fzf-directory support fzf file preview**	For **fzf** and **fzf-directory**: With optional prefix (eg. **C-u**) the currently selected file content or attribute is shown using the preview command identified by the 🎛 **'fzf/ args-for-preview'** user-option. By default that shows the file content with cat, but that can be customized to use other mechanisms.		
Open file searched by fzf in specified directory	• `<f11> M-z M-d` • `<f11> f d`	(**fzf-directory** &optional WITH-PREVIEW)	Prompt for a directory to perform the fzf file search, then open selected file inside current window. Directory prompt uses current completion mode. See Σ **Completion/Input**.
Open fzf searched file in current or specified directory using ivy I/F	`<f11> f c`	(**counsel-fzf** &optional INITIAL-INPUT INITIAL-DIRECTORY FZF-PROMPT)	Open a file selected by ivy-style prompt using a fzf shell command. • With **C-u** prefix argument first prompts for the directory to perform the fzf search. ⚠️ Much slower than (fzf) for large directories because counsel captures fzf output before showing it.
Switch buffer with fzf See also: Σ **Buffers**	`<f11> b z`	(**fzf-switch-buffer**)	Switch buffer in current window by selecting it with fzf. • Uses the fzf command line utility for fast & flexible search. 📦 Requires the fzf.el external package activated by **pel-use-fzf** .
Search/open Git repo member files with fzf	`<f11> f g`	(**fzf-git-files**)	Search files committed current Git repository with fzf and open user selected file.
Search/open committed file in Git repo directory tree with fzf	`<f11> f G`	(**fzf-git**)	Search all files in current Git repository with fzf and open user selected file.
Search/open committed file in Mercurial repo tree with fzf	`<f11> f h`	(**fzf-hg-files**)	Search files committed current Mercurial repository with fzf and open user selected file.
Search/open file in Mercurial repo directory tree with fzf	`<f11> f H`	(**fzf-hg**)	Search all files in current Mercurial repository with fzf and open user selected file.
Search/open file in current projectile project with fzf. See Σ𝑥 Projectile	`<f11> f <f8>` `<f8> M-z`	(**fzf-projectile** &optional WITH-PREVIEW)	Search all files in current projectile project with fzf and open selected file. With **C-u** show file preview. 📦 Requires the fzf.el external package activated by **pel-use-fzf** 📦 Requires the projectile external package activated by **pel-use-projectile**
Grep search files with fzf for specified regex	`<f11> g s`	(**fzf-grep**)	Prompt for string to search and file grep selection expression, show grep results in a fzf session, select appropriate line to open the specific file at appropriate line.
Grep search files with fzf for specified regex in specified directory	`<f11> g S`	(**fzf-grep-in-dir**)	Prompt for directory, then for string to search and file grep selection expression, show grep results in a fzf session, select appropriate line to open the specific file at appropriate line.
Grep search Git repo member files with fzf for specified regex	`<f11> g G`	(**fzf-git-grep**)	Prompt for string to search and file grep selection expression, show grep results over current Git repo searched in a fzf session, select appropriate line to open the specific file at appropriate line. 🚧⚠️ This command does not seem to work properly, it searches but does not always open the file.
Open Dired (Directory Editor) See also: Σ **Dired**	When "opening" (visiting) a directory Emacs opens a buffer in Dired mode, that looks like a ls -l output, which allows several operations. If you specify a directory path to `C-x C-f` then Dired-mode is used. You can also use the following commands to open buffer in Dired mode. • Prompt input completion can be changed for these. See Σ **Completion/Input** ☝ It's also possible to browse a file directory tree with **file tree browsers,** like NeoTree and ztree (see below), or with Σ **Speedbar** .		
Open a directory editor	• `C-x d` • `⌘-D`	• (**dired** DIRNAME &optional SWITCHES) • (**ido-dired**)	Opens a Dired-mode buffer on the specified directory. Prompt for the directory name. PEL activates ido when the **pel-use-ido-mode** user option is set to **t**.
Run Dired in other window	`C-x 4 d`	(**dired-other-window**)	Opens a Dired-mode buffer on the specified directory inside another window. • Prompt for the directory name.
List Directory	`C-x C-d`	(**list-directory** DIRNAME &optional VERBOSE)	Display a list of files in or matching DIRNAME, a la 'ls'. • DIRNAME is globbed by the shell if necessary. • Prefix arg (**C-u**) means supply -l switch to 'ls'.
Jump to file entry in dired buffer ★★ ☝Leaves point on the file jumped to, allowing immediate Dired action, eg.: `C-x C-j R` **renames** the file.	`C-x C-j`	(**dired-jump** &optional OTHER-WINDOW FILE-NAME)	Jump to Dired buffer corresponding to current buffer. • If in a file, Dired the current directory and move to file's line. • If in Dired already, pop up a level and goto old directory's line. • In case the proper Dired file line cannot be found, refresh the dired buffer and try again. • When OTHER-WINDOW is non-nil, jump to Dired buffer in other window. • When FILE-NAME is non-nil, jump to its line in Dired. • **Interactively** with prefix argument, read FILE-NAME.
Activating URLs to browse and open files	Emacs provides the **goto-url-mode** and the **goto-url-prog-mode** that turn URLs found in the current buffer into clickable buttons. • Once the mode is active the following key sequences are available wheel point is over a URL button: • `C-c RET` or the **mouse to click** on the button. 🎛 Customization group: **goto-address** • If the URL is an email address a buffer to write an email to that address opens. • If the URL is a web or FTP address the system browser is invoked to open the address. • `C-c C-n` : move point to the end of the next URL in the buffer. • `C-c C-p` : move point to to the previous URL in the buffer. • `C-c C-f` : download the file identified by the URL into a local temporary file and visit the file. See (pel-open-url-at-point) above.		
Toggle goto-address-mode	`<f11> f u`	(**goto-address-mode** &optional ARG)	Minor mode to buttonize URLs and e-mail addresses in the current buffer. With a prefix argument ARG, enable the mode if ARG is positive, and disable it otherwise.
Toggle goto-address-prog-mode	`<f11> f U`	(**goto-address-prog-mode** &optional ARG)	Like 'goto-address-mode', but only for comments and strings.
Open the URL (email or web page)	`C-c RET`	(**goto-address-at-point** &optional EVENT)	Open the URL at point. If URL is a web page: open it in a browser. • If URL is a mail address: Send mail to address at, around point or before.
Move to end of next URL in buffer See also: Σ **Navigation**	`C-c C-n` `<f6> C-n`	(**pel-goto-next-url**)	Move point forward to the end of the next URL located in the current buffer. • The global `<f6> C-n` key binding activates the goto-address-mode if it is not already active.
Move to beginning of previous URL in buffer	`C-c C-p` `<f11> C-p`	(**pel-goto-previous-url**)	Move point backward to the beginning of the previous URL located in the current buffer. • The global `<f6> C-p` key binding activates the goto-address-mode if it is not already active.
Insert text of another file at point	The following commands can be used to insert text from other files at point in the current buffer.		
Insert file at point	• `C-x i` • `<f11> f i`	• (**insert-file** FILENAME) • (**ido-insert-file**)	Insert contents of file FILENAME into buffer after point. • Set mark after the inserted text.
Insert file literally at point	`<f11> f I`	(**insert-file-literally** FILENAME)	Insert contents of file FILENAME into buffer after point with no conversion. • Set mark after the inserted text.

Operation	Keystroke	Function	Note
Write text into specified file	The following commands can be used to write text selected from current buffer into specified file.		
Write region text to file	`<f11> f w`	(**write-region** START END FILENAME &optional APPEND VISIT LOCKNAME MUSTBENEW)	Write current region into specified file. • Prompts for the specified file.
Append region text to file	`<f11> f W`	(**append-to-file** START END FILENAME)	Append the contents of the region to the end of file FILENAME. • Prompts for the specified file.
Set file mode	`<f11> f m`	(**set-file-modes** FILENAME MODE)	Set mode bits of file named FILENAME to MODE (an integer). • Only the 12 low bits of MODE are used. • Prompts for file name and then for chmod-like file mode value.
Reverting Files	If the file's content changed on the disk and you want to refresh the Emacs buffer visiting that file, you need to "revert" the file. • If you want to use Emacs to monitor the content of a file that is continuously modified by an external process (like a log file) set the ***revert-without-query*** variable to a list of regular expressions describing the field it'll apply to. • You can also activate the auto-revert mode for the current buffer or globally and restart its timer.		
Revert a buffer See also: **Diff & Merge**	• `<f11> f r f` • `⌘-u`	(**revert-buffer** &optional IGNORE-AUTO NOCONFIRM PRESERVE-MODES)	Replace current buffer text with the text of the visited file on disk. • This undoes all changes since the file was visited or saved. • With a prefix argument, offer to revert from latest auto-save file, if that is more recent than the visited file. • This is also the command to use to reload a file that was modified on the file system. ☝ Use **ediff-current-file** to see difference between the buffer and its disk file, with: `<f11> e b f`.
Toggle auto-revert mode	`<f11> f r a`	(**auto-revert-mode** &optional ARG)	Toggle reverting buffer when the file changes (Auto-Revert Mode). With a prefix argument ARG, enable Auto-Revert Mode if ARG is positive, and disable it otherwise. • Auto-Revert Mode is a minor mode that affects only the current buffer. When enabled, it reverts the buffer when the file on disk changes. • When a buffer is reverted, a message is generated. This can be suppressed by setting 'auto-revert-verbose' to nil.
Toggle auto-revert tail mode See also: **Scrolling**	• `<f11> | t` • `<f11> f r t`	(**auto-revert-tail-mode** &optional ARG)	Toggle reverting tail of buffer when the file grows. • With a prefix argument ARG, enable Auto-Revert Tail Mode if ARG is positive, and disable it otherwise. • When Auto-Revert Tail Mode is enabled, the tail of the file is constantly followed, as with the shell command 'tail -f'. This means that whenever the file grows on disk (presumably because some background process is appending to it from time to time), this is reflected in the current buffer. • You can edit the buffer and turn this mode off and on again as you please. But make sure the background process has stopped writing before you save the file!
Cancel/restart auto-revert timer	`<f11> f r SPC`	(**pel-auto-revert-set-timer**)	Restart or cancel the timer used by Auto-Revert Mode. If such a timer is active, cancel it. • Start a new timer if Global Auto-Revert Mode is active or if Auto-Revert Mode is active in some buffer. • Restarting the timer ensures that Auto-Revert Mode will use an up-to-date value of '***auto-revert-interval***'(which is normally 5 seconds by default). 💻 : pel-auto-revert-set-timer is a thin wrapper over auto-revert-set-timer that displays a warning if executed when the buffer is not already in auto-revert-mode. It also displays the value of *auto-revert-interval* when auto-revert-set-timer is executed.
Saving Files ☝ **To rename** one file use one of: • **C-x C-j R** • `C-x C-w` ☝ **To rename several** files use: • Wdired (**Dired**)	Use the following commands to save the content of a buffer to a filesystem file. • PEL supports the following controllable actions on file save. Each of these actions are activated via an action-specific PEL user-option, and can temporarily be disabled with a command for the file in the current buffer. The actions and their associated user-option and command are listed here: **Action** / **Activating user-option** / **Overriding command** / **Key Sequence** • Delete trailing space and lines on save / pel-delete-trailing-whitespace / pel-toggle-delete-trailing-space-on-save / `<f11> M-W` • override it for some major modes: / pel-modes-preventing-delete-trailing-whitespace • Update time stamp on save / pel-update-time-stamp / pel-toggle-update-time-stamp-on-save / `<f11> f t T` • Update copyright notice on save / pel-update-copyright / pel-toggle-update-copyright-on-save / `<f11> M-C`		
Save file to disk	• `C-x C-s` • `⌘-s`	(**save-buffer** &optional ARG)	Save current buffer to associated file. By default, it makes the previous version into a backup file if previously requested or if this is the first save. On macOS in graphics mode only: `⌘-s` brings a OS file-save dialog. • With `C-u`: marks this version to become a backup when the next save is done • With `C-u C-u`: makes the previous version into a backup file • With `C-u C-u C-u:` marks this version to become a backup when the next save is done, and makes the previous version into a backup file. • With prefix 0: never make the previous version into a backup file. ⚠ Save and activated on-file-save actions only occur when the buffer is in "changed" status. Use `M-~` to flip that status to force an action when it has just been activated.
Save all/some files	`C-x s`	(**save-some-buffers** &optional ARG PRED)	Prompt for files that are modified. Options: • **y** : save **n** : don't save • **C-r** : look at the buffer in question. **d** : view differences with diff-buffer-with-file
Write buffer to specified file ☞ **Save As/Rename**	`C-x C-w`	• (**write-file** FILENAME &optional CONFIRM) • (**ido-write-file**)	Similar to "Save-As": prompt for the filename. • Can also be yanked in the mini buffer, use `M-n` to edit it. 💡 Use that command to **rename** the file.
Changed current buffer changed state	`M-~`	(**not-modified** &optional ARG)	Mark current buffer as unmodified, not needing to be saved. • With `C-u` prefix ARG, mark buffer as modified, so `C-x C-s` will save.
Toggle copyright update on save	`<f11> M-@`	(**pel-toggle-update-copyright-on-save** &optional GLOBALLY)	Toggle copyright update on file save and display current state. • By default change behaviour for local buffer only. • When GLOBALLY argument is non-nil, using any prefix argument, change it for all buffers for the current Emacs editing session (the change does not persist across Emacs sessions). • To modify the global state permanently modify the customized value of the 'pel-update-copyright' user option via the 'pel-pkg-for-filemng' group customize buffer with **<f11> f <f2> 1**. 🔄 This command is only available when the **pel-update-copyright** is set to **t**.
Toggle delete trailing space on save See also: **Whitespace**	• `<f11> M-W` • `<f11> t w M-W`	(**pel-toggle-delete-trailing-space-on-save** &optional GLOBALLY)	Toggle deletion of trailing spaces on file save and display current state. • By default change behaviour for local buffer only. • When GLOBALLY argument is non-nil, using any prefix argument, change it for all buffers for the current Emacs editing session (the change does not persist across Emacs sessions). 🎛 Trailing whitespace deletion is automatically activated on file save when the **pel-delete-trailing-whitespace** user-option is set to **t**. Use this command to de-activate it or re-activate it. • To modify the global state permanently modify the customized value of the '**pel-delete-trailing-whitespace**' user option via the '**pel-pkg-for-filemng**' group customize buffer with **<f11> f <f2> 1**.
Inserting & Automatically Updating Copyrights	Emacs has built-in support for insertion and update of copyright notices inside files. It provides 2 commands to insert or update the file's copyright notice. • The copyright notice can be automatically updated by adding the **copyright-update** function to the list of **before-save-hook** variable with the following code: `(add-hook 'before-save-hook 'copyright-update)` ⚠ To be automatically updated, the copyright notice must be placed within an area at the beginning of the file specified by the value of the **copyright-limit** user-option, normally defined as the first 2000 characters.		
Insert copyright notice at point	`<f11> i C`	(**copyright** &optional STR ARG)	Insert a copyright by $ORGANIZATION notice at cursor. ➨ See also: **Inserting Text** • If the ORGANIZATION environment variable is not available, Emacs prompts for it.
Update file's copyright notice	`M-x copyright-update`	(**copyright-update** &optional ARG INTERACTIVEP)	Update copyright notice to indicate the current year. • With prefix ARG, replace the years in the notice rather than adding the current year after them. If necessary, and 'copyright-current-gpl-version' is set, any copying permissions following the copyright are updated as well. ⚠ Even when used interactively copyright-update does not warn if there is no copyright in the current buffer to update. It does not create a missing notice. ☝ If you want to be prompted automatically to update an existing but out-of-date copyright notice, write the following inside your init.el file: `(add-hook 'before-save-hook 'copyright-update)`

Operation	Keystroke	Function	Note

Automatic File Time Stamp on file save
 	
References:
• **TimeStamps @ EmacsWiki**
• **Change time stamp format in:**
  • **markdown file**
  • **reStucturedText file**
 	
☝️Use **ielm elisp shell** to test the time format using the **format-time-string** function.
 	
See also: **Σ Inserting Text**
			Emacs has a built-in **automatic time-stamping of files.** It must be activated by adding the **time-stamp** function to the **before-save-hook** variable.
			This can either be done via Emacs customization system or explicitly inside your init file with the following code:
			`(add-hook 'before-save-hook 'time-stamp)`
			• The time stamp will be added to files that contain, inside their first 8 lines, a line that looks like one of the following:
			  • `Time-stamp: <>`
			  • `Time-stamp: " "`
			☝️ You can, however change these defaults and get Emacs to update all sorts of time stamp formats, even inside source code statements:
			🎛️ Emacs controls automatic insertion of timestamp with the following variables:
			• **time-stamp-pattern** consists of 4 parts, each one controlled by a variable:
			  • **time-stamp-line-limit** : identifies where in the file the time stamp can be located. Defaults to 8: the first 8 lines.
			  • **time-stamp-start:** identifies the text pattern that precedes the time stamp.
			  • **time-stamp-end**: identifies the end of the time stamp.
			  • **time-stamp-format** specifies the format of the time stamp.
			  • Something like `"%:y-%02m-%02d %02H:%02M:%02S %u"` to specify the date and time in ISO format, with the user login's name.
			• **time-stamp-time-zone** specifies the time zone selection:
			  • **nil**: Emacs local time, **t**: Universal time, **wall** : system wall clock time, **TZ** : controlled by a TZ environment variable
			The **time-stamp-format** and **time-stamp-time-zone** variables can be set in your init file or via the Emacs customization system.
			• They are defined in the **time-stamp** customization group.
			• ☝️ To change the format or the pattern preceding or after the automatically updated time stamp, it is best to use file local variables: this will allow automatic time stamp updates in files with various formats. As an example, see the top and end of the PEL manual raw format file.
			⚠️ By default, the time-stamp string must be placed within the **first 8 lines** of the file, otherwise it will not be updated automatically.
			• If you want it located somewhere else in your file set the **time-stamp-line-limit** file local variable.
			🎛️ PEL provides the extra user-option to control the automatic generation of time-stamps:
			• **pel-update-time-stamp** user-option controls whether time-stamps are automatically updated when saving file(s) where a valid time-stamp corresponding to Emacs settings as described above. Set it to **t** (the default) to allow automatic time stamp updates. Set it to nil to prevent them. You can also toggle it globally for the current editing session by using the **`<f11> f M-t`** key sequence.
			☝️ To insert a non-updatable time stamp, the PEL package provides a set of text insert commands which include inserting a time stamp .
			• See the **Σ Inserting Text** table for the appropriate commands.

Show the values of time stamp controlling variables	**`<f11> f t ?`**	(**pel-time-stamp-control-show-info** &optional APPEND)	Display buffer current time stamp control variables and their state. The information is shown inside a *pel-time-stamp-info* help buffer. The user-options listed are buttons you can use to get more info and access the customization buffers.

Example 1:
 	
The timestamp used by the PEL source code.
In this case the values of all customizable user-options are specified by customization.
	The format of the timestamp used by PEL source code is the following (all in 1 line):
	
	;; Time-stamp: <2025-11-12 22:11:29 EST, updated by Pierre Rouleau>
	
	In such a file, the information shown by the command is shown here: ➽➽➽
			`-`**`---Automatic File Time Stamp Control from *scratch* --- Wednesday, November 12, 2025 @ 22:11:44 -----`**
			
			PEL provides control of the hook logic required to automate the update
			of time stamp when a file is saved; it controls it with the value of the
			'pel-update-time-stamp' option. On startup, PEL sets up the hook for a
			function that updates time stamp when it is non-nil.
			
			- You can change this dynamically with 'pel-toggle-update-time-stamp-on-save'
			  command, bound to `<f11> f t T`.
			- It also updates 'time-stamp-active' for consistency.
			- When Emacs starts, PEL set 'time-stamp-active' to the value of
			  'pel-update-time-stamp' to ensure consistency.
			- Note that 'time-stamp-toggle-active' (bound to `<f11> f t M-T`)
			  only toggle 'time-stamp-active' which affects whether time stamp is updated
			  by the 'time-stamp' command (bound to `<f11> f t t`.
			- For a time stamp to be updated on file save, both variables must be non-nil.
			
			- pel-update-time-stamp : t : Time stamp updated on file save in this session.
			- time-stamp-active : t : Only controls whether time-stamp command updates the time stamp.
			
			***Time Stamp Location and Format Control:**
			
			The location and format of the time stamp is either controlled by the single
			`time-stamp-pattern' or all of the other 4 user-options, all 4 of them
			otherwise you risk using a mix of what you want and what was already active.
			
			- time-stamp-pattern : nil
			- time-stamp-line-limit : 8
			- time-stamp-start : "Time-stamp:[ ]+\\\\?[\"<]+"
			- time-stamp-end : "\\\\?[\">]"
			- time-stamp-format : "%Y-%02m-%02d %02H:%02M:%02S %Z, updated by %U"
			
			-UUU:%*- F1 *pel-time-stamp-info* All (1,0) (Help WK Anzu) 22:15 1.12 ----------

Example 2:
 	
Package-Version timestamp for MELPA compilation package file.
 	
This example is taken from **tbindent.el**
	Use file-local variable setting (see **Σ File/Dir Variables**) to set the 4 user-options required to specify the time stamp format required for a Package-Version field:
	
	;; Author: Pierre Rouleau <prouleau001@gmail.com>
	;; Maintainer: Pierre Rouleau <prouleau001@gmail.com>
	;; URL: https://github.com/pierre-rouleau/tab-based-indent
	;; Created : Monday, November 10 2025.
	;; Version: 0.1
	;; Package-Version: 20251112.1730
	;; Keywords: convenience, languages
	;; Package-Requires: ((emacs "24.3"))
	
	...
	
	;; Local variables:
	;; time-stamp-format: "%Y%02m%02d.%02H%02M"
	;; time-stamp-start: "Package-Version:[\t]+\\\\?"
	;; time-stamp-end: "\n"
	;; time-stamp-line-limit: 15
	;; End:
	
	The information shown for this file is: ➽➽➽
			`----Automatic File Time Stamp Control from tbindent.el --- Wednesday, November 12, 2025 @ 22:16:55 -----`
			
			PEL provides control of the hook logic required to automate the update
			of time stamp when a file is saved; it controls it with the value of the
			'pel-update-time-stamp' option. On startup, PEL sets up the hook for a
			function that updates time stamp when it is non-nil.
			
			- You can change this dynamically with 'pel-toggle-update-time-stamp-on-save'
			  command, bound to `<f11> f t T`.
			- It also updates 'time-stamp-active' for consistency.
			- When Emacs starts, PEL set 'time-stamp-active' to the value of
			  'pel-update-time-stamp' to ensure consistency.
			- Note that 'time-stamp-toggle-active' (bound to `<f11> f t M-T`)
			  only toggle 'time-stamp-active' which affects whether time stamp is updated
			  by the 'time-stamp' command (bound to `<f11> f t t`.
			- For a time stamp to be updated on file save, both variables must be non-nil.
			
			- pel-update-time-stamp : t : Time stamp updated on file save in this session.
			- time-stamp-active : t : Only controls whether time-stamp command updates the time stamp.
			
			***Time Stamp Location and Format Control:**
			
			The location and format of the time stamp is either controlled by the single
			`time-stamp-pattern' or all of the other 4 user-options, all 4 of them
			otherwise you risk using a mix of what you want and what was already active.
			
			- time-stamp-pattern : nil
			- time-stamp-line-limit : 15
			- time-stamp-start : "Package-Version:[ ]+\\\\?"
			- time-stamp-end : "
			"
			- time-stamp-format : "%Y%02m%02d.%02H%02M"
			
			-UUU:%*- F1 *pel-time-stamp-info* All (1,0) (Help WK Anzu) 22:17 1.36 ----------

Toggle timestamp update on save	**`<f11> f t T`**	(**pel-toggle-update-time-stamp-on-save** &optional GLOBALLY)	Toggle time-stamp update on (current buffer's) file save and display current state. • By default change behaviour for local buffer only. • When GLOBALLY argument is non-nil, using any prefix argument, change it for all buffers for the current Emacs editing session (the change does not persist across Emacs sessions). • Also update the buffer-local or global value of '**time-stamp-active**' for consistency. • To modify the global state permanently modify the customized value of the '**pel-update-time-stamp**' user option via the '**pel-pkg-for-filemng**' group customize buffer with **<f11> f <f2> 1**.
Update file time stamp	**`<f11> f t t`**	(**time-stamp**)	Force update the time stamp string(s) in the current buffer. Updates a time stamp of format recognized by *Emacs current settings* even when automatic time-stamp update is off. • More information about the "*Emacs current settings*" in the description block above.
Toggle time stamp modification done by time-stamp command	**`<f11> f t M-T`**	(**time-stamp-toggle-active** &optional ARG)	Toggle '**time-stamp-active**', setting whether **`<f11> f t t`** updates a buffer: it sets the value of the **time-stamp-active** user-option. With ARG, turn time stamping on if and only if arg is positive. ⚠️ This only controls whether time-stamp will update the time stamps in the current buffer. It does not have an impact on whether the time stamps are updated when the file is saved as this requires logic to set the hook. Since PEL does control the hook as specified by the value of the '**pel-update-time-stamp**' user-option, then PEL stores the value of '**pel-update-time-stamp**' into 'time-stamp-active'

Operation	Keystroke	Function	Note
RFC-Mode	Browsing and reading RFC Files with the following **rfc-mode** commands. 📦 Requires **rfc-mode** activated by **pel-use-rfc**. • Use `<f11> B <f2> 1` to access its PEL customization. While viewing a RFC, use `n` and `p` to move to the next or previous RFC section.		
Read a specific RFC	`<f11> B r`	(**rfc-mode-read** NUMBER)	Read the RFC document NUMBER. Offer the **number at point** as default.
Browse RFCs	`<f11> B R`	(**rfc-mode-browse**)	Browse through all RFC documents referenced in the index.
Directory Tree Browsers	Emacs supports several mechanisms to browse file directories. This includes: • Emacs built-in **Dired** directory editor, along with several extensions. You can have several different Dired buffers in an Emacs session. • The Emacs built-in **Speedbar** and its extensions. There can only be one instance of a Speedbar buffer and that can be inside another frame. • Several other external packages: **dir-treeview**, **Neotree**, **treemacs**, **lsp-treemacs** and **Ztree** • Use `<f11> B <f2> 1` to access their PEL customization and `<f11> B <f3>` to access the customization of these packages.		
dir-treeview	📦 The **dir-treeview** external package provide a simple to use expandable directory tree view in a buffer. PEL activates it when **pel-use-dir-treeview** is set to **t**. Access its configuration via `<f11> B <f3> 1`		
Browse home (or default) directory tree	`<f11> B D`	(**dir-treeview**)	Display the default directory tree inside the current (or new) *Dir Treeview* buffer. • Open the directory identified by the **dir-treeview-default-root** user-option which defaults to the home directory.
Browse selected directory tree	`<f11> B d`	(**dir-treeview-open** &optional DIR)	Prompt for directory, then display its directory tree inside the current (or new) *Dir Treeview* buffer. • The pro pomp proposes the **dir-treeview-default-root** user-option which defaults to the home directory.
View Directory Tree with NeoTree	📦 The **NeoTree** external package provides a Vim-NerdTree like tree-view of a directory with expansion/collapse. PEL activates it when **pel-use-neotree** is set to **t**. • `<f11> B N <f2>` opens the PEL customization group to set **pel-use-neotree**. • `<f11> B N <f3>` prompts, select neotree to open the neotree customization group. ☛ There is only **one** NeoTree window. It is a **dedicated window**. ☛ Icons used in the tree can be changed: • In text mode set pel-neotree-font-in-terminal to arrows to use arrows instead of '+'. • In graphics mode, if pel-neotree-font-in-graphics is set to icons then the icons provided by **all-the-icons package** is used. ⚠️ However, once PEL has installed the package it does not install the fonts. You must install the fonts manually by executing: `M-x all-the-icons-install-fonts`		
View directory tree with NeoTree	`<f11> B N N`	(**neotree-toggle**)	Toggle show/hide the NeoTree window. In the NeoTree buffer the following keys are available: • `n` next line, `p` previous line. > end of buffer, < top buffer • `SPC` or `RET` or `TAB` : Open current item if it is a file, Fold/Unfold current item if it is a directory. • `U` Go up a directory. `g` Refresh • `A` Maximize/Minimize the NeoTree Window • `H` Toggle display hidden files. Controlled by **neo-hidden-regexp-list** user option. • `O` Recursively open a directory • `C-c C-n` Create a file or create a directory if filename ends with a '/' • `C-c C-d` Delete a file or a directory. `C-c C-r` Rename a file or a directory. • `C-c C-c` Change the root directory. `C-c C-p` Copy a file or a directory.
Open NeoTree for dir of current buffer	`<f11> B N F`	(**neotree-find** &optional PATH DEFAULT-PATH)	Open a NeoTree window using the directory of the current buffer. No prompt.
Open NeoTree for specified directory	`<f11> B N D`	(**neotree-dir** PATH)	Prompt for a directory. Open a Neotree window for that directory.
Close NeoTree window	`<f11> B N H`	(**neotree-hide**)	Close the NeoTree window.
Show NeoTree window	`<f11> B N S`	(**neotree-show**)	Show the NeoTree window.
Treemacs • Manipulate directory trees associated as projects/workspaces • Manipulate the directories and files ★★ See: **Treemacs**	📦 The **treemacs** and **lsp-treemacs** provides workspace/project oriented tree-based view with expansion/collapse and actions of directories and files. PEL activates treemacs when the **pel-use-treemacs** or **pel-use-lsp-treemacs** user-option is turned on (set to **t**). 🎛 Treemacs has a large number of user-options in the **treemacs** customization group and sub-groups. • Use `<f11> B <f2> 3` to access its PEL customization for it. • and `<f11> B <f3> 3` to access its customization group. On PEL, open (or close) the treemacs buffer with the `<f11> B T` key sequence. • In graphics mode the mouse provides access to most commands. • In terminal (and graphics) mode when pain is inside the treemacs dedicated window, the treemacs major mode key-bindings, listed below, are available. The treemacs-mode and extensions have an extensive command set. See **Treemacs** for the complete list		
Open/close treemacs	`<f11> B T`	(**treemacs**)	Initialise or toggle treemacs. See **Treemacs** for treemacs-mode commands. • If the treemacs window is visible hide it. • If a treemacs buffer exists, but is not visible show it. • If no treemacs buffer exists for the current frame create and show it. • If the workspace is empty additionally ask for the root path of the first project to add.
View Directory Tree with ZTree	📦 The **ztree** external package provides a text-based tree-view of a directory with expansion/collapse. 🎛 PEL ztree customization: • `<f11> B <f2>` opens the PEL customization group (select the ztree subgroup) . See also: **Customize**. • PEL activates it when **pel-use-ztree** is set to **t**. • Modify one of the following PEL provided customization user options: • **pel-ztree-dir-move-focus** : set to **t** to move focus to new entry when <RET> is typed. • **pel-ztree-dir-filter-list** : add a list of regexp to ignore more file. Do not enter quote for string. For example, to ignore the .pyc files, enter **^.*pyc** on a line. • **pel-ztree-show-filtered-files** : set to **t** to display filtered files until `H` is typed. Normally they are not shown until `H` is typed. • `<f11> B <f3>` prompts, select ztree to open the ztree customization group itself. • Execute `M-x pel-init` after settling and applying new values to activate the new values, or restart Emacs.		
View directory as tree with ztree-dir	`<f11> B Z`	(**ztree-dir** PATH)	Open an interactive buffer with the directory tree of the PATH given. • Opens the tree buffer in the current window. • There can be several buffers with different ztree-dir trees.
		In the Ztree Dir buffer the following keys are available: • **>** : narrow/display directory on current line **<** : widen/display parent directory • `d` : Open Dired at point. • `H` : toggle display of filtered files. Controlled by regexp in the **ztree-dir-filter-list** user option. • `x` : Toggle expand/collapse of all nodes of the subtree. • ⚠️ Use `x` with care! On large directory trees it takes a long time. I have seen Emacs hang when typing `x` again during that time. 🚧 Investigate.	

Operation	Keystroke	Function	Note
Searching/Finding Files See also: • Σ **Help/Info** • Σ **Dired**	The following commands can be used to search for file by name or content. ☝ You can also use the fuzzy file search see fzf above. • See: Video: .Emacs #6 : searching and finding files. ☝ Use **man** to get more information, • on locate: `<f11> ? m` locate • on find: `<f11> ? m` find ☝ You can manipulate the result in Dired with Dired commands. For instance type `(` to toggle the display of more than the file names.		
Search for file with locate	`<f11> f L`	(**locate** SEARCH-STRING &optional FILTER ARG)	Prompt for a search pattern and search for filenames using the system **locate** command line utility through the sell to search a database of all pathnames that match the specified search pattern. The database is recomputed periodically. • The search result is shown in a '*Locate*' buffer. • With prefix arg ARG, prompt for the exact shell command to run instead. This way you can specify options to the locate command line utility.
		(**counsel-locate** &optional INITIAL-INPUT)	Call a "locate" style shell command with counsel listing and completion user-interface. • INITIAL-INPUT can be given as the initial minibuffer input. This binding activated when the **pel-use-counsel** user-option is turned on. When **pel-use-ivy-hydra** user-option is set you can activate the **ivy-hydra** with **C-o**. When Hydra is active, minibuffer editing is disabled and menus display short aliases: **Short Normal Command name** o **C-g** keyboard-escape-quit j **C-n** ivy-next-line k **C-p** ivy-previous-line h **M-<** ivy-beginning-of-buffer l **M->** ivy-end-of-buffer d **C-m** ivy-done f **C-j** ivy-alt-done g **C-M-m** ivy-call u **C-c C-o** ivy-occur
Run grep via find See also: Σ **Grep**	• `<f11> f F g` • `<f11> g f`	(**find-grep** COMMAND-ARGS)	Run grep via find, with user-specified args COMMAND-ARGS. • Collect output in a buffer. • While find runs asynchronously, you can use the **`C-x ``** command to find the text that grep hits refer to. • This command uses a special history list for its arguments, so you can easily repeat a find command.
Search for files with 'find' and open Dired buffer	`<f11> f F d`	(**find-dired** DIR ARGS)	Prompts for the root to search from, and a **find** command to search for files with the Unix find. • Specify the arguments for the find command. • For example, to perform a case insensitive search for all .h files, use: `-iname "*\.h"` • Opens a Dired-mode buffer and show the files found in there.
Search directory for files and open Dired buffer for those See: Σ **Dired**	• `<f11> f F n` • `<f11> f n`	(**find-name-dired** DIR PATTERN)	Search DIR recursively for files matching the globbing pattern PATTERN, and run Dired on those files. • PATTERN is a shell wildcard (not an Emacs regexp) and need not be quoted. • The default command run (after changing into DIR) is: `find . -name 'PATTERN' -ls`
Find files in a directory and open Dired output	`<f11> f F h`	(**find-grep-dired** DIR REGEXP)	Find files in DIR that contain matches for REGEXP and start Dired on output. The command run (after changing into DIR) is: find . \(-type f -exec 'grep-program' 'find-grep-options' -e REGEXP {} \; \) -ls where the first string in the value of the variable '**find-ls-option**' specifies what to use in place of "-ls" as the final argument.
Find Emacs Lisp files in directory tree	`<f11> f F l`	(**find-lisp-find-dired** DIR REGEXP)	Find Emacs Lisp files in DIR, matching REGEXP. • Open *Find Lisp Dired* buffer on output.
List all broken symbolic links inside a directory tree	`<f11> f s`	(**pel-show-broken-symlinks**)	Find and list broken symbolic links in directory tree. • Prompts for a directory and list found broken symlinks in the special *Broken Symlinks* buffer if any found. ⚠ This can take a long time if the directory tree contains a large number of files. Emacs operation are blocked during that time.

File Management — References

Topic & Link	Description
Emacs Display - Mode Line	Read first. Describes what the Emacs mode line displays.
GNU Emacs Manual - File Handling	Describes how to open and deal with files and directories in Emacs.
GNU EMACS Manual - Interactive Do	Describes the ido-mode, a nice addition that helps with completing file names at prompts.
Display path of file in status bar	In graphics mode, display the buffer name and the full path file in parenthesis inside the frame title bar.
How do I rename an open file in Emacs?	
Find files faster with the recent files package	Mickey Petersen article describing the recent file feature. PEL ido-recentf-open is taken from Mickey Peterson code.